講義 8：模組化（Modularization）——FORM / USING / CHANGING（授課順序：接在講義 8a 之後）

> 對應練習：ex08 | 答案程式：`ZR_TR08_MODULARIZE`

本講重點

- 為什麼要模組化：主流程一眼看懂、邏輯可重複使用
- `FORM ... ENDFORM` 定義副程式、`PERFORM` 呼叫
- 補充（選修）：跨程式呼叫 FORM（`PERFORM ... IN PROGRAM`）與 Subroutine Pool
- `USING`（輸入）與 `CHANGING`（輸入兼輸出）的分工
- 參數要加型別（TYPE）；區域變數 vs 全域變數
- 傳統報表的標準骨架

1. 為什麼要模組化

所有邏輯塞在 START-OF-SELECTION 裡，兩百行之後沒人讀得懂。把「取數」「加工」「輸出」各自包成 FORM，主流程剩三行，讀程式像讀目錄：

```
START-OF-SELECTION.
  PERFORM get_data.
  PERFORM process_data.
  PERFORM display_data.
```

> 定位說明：FORM（subroutine）是傳統報表的模組化手段，本專案既有程式（如 ZDQM 系列）大量使用，必須熟練。跨程式共用的邏輯用 Function Module（講義 15）；新世代開發用 Class Method（OOP 課程）。三者是同一件事在不同時代的答案。

2. FORM 與 PERFORM

```
* 呼叫（事件區）
START-OF-SELECTION.
  PERFORM say_hello.

* 定義（放在程式最後面）
FORM say_hello.
  WRITE / 'Hello from FORM!'.
ENDFORM.
```

- FORM 定義習慣**集中放在程式尾端**（所有事件之後），與事件區隔開；SE38 內雙擊 FORM 名可直接跳轉。
- 定義了沒被呼叫的 FORM 不會執行；PERFORM 一個不存在的 FORM 是編譯錯誤。

3. 參數：USING 與 CHANGING

FORM 可以宣告參數，呼叫端與定義端順序一一對應：

```
* 呼叫端
PERFORM calc_grade USING    gs_student-score
                   CHANGING gv_grade.

* 定義端：參數務必加 TYPE
FORM calc_grade USING    iv_score TYPE i
                CHANGING cv_grade TYPE c.
  IF iv_score >= 80.
    cv_grade = 'A'.
  ELSEIF iv_score >= 60.
    cv_grade = 'B'.
  ELSE.
    cv_grade = 'C'.
  ENDIF.
ENDFORM.
```

區段	語意	慣例前綴
`USING`	輸入：FORM 只讀它	`iv_` / `is_` / `it_`
`CHANGING`	輸入兼輸出：FORM 會改它，改動帶回呼叫端	`cv_` / `cs_` / `ct_`

技術上 USING 預設也是「傳參考」，在 FORM 裡改它其實改得到呼叫端——但**團隊紀律是 USING 一律當唯讀**，要改就放 CHANGING，讓介面誠實。想強制唯讀可寫 `USING VALUE(iv_score) TYPE i`（傳值複本）。

參數不加 TYPE 雖然能編譯（成為任意型別），但失去檢查、埋下錯誤——**一律加 TYPE**。內表參數這樣寫：

```
FORM show_list USING it_students TYPE tt_student.
  DATA ls_student TYPE ty_student.        " 區域 work area
  LOOP AT it_students INTO ls_student.
    WRITE: / ls_student-id, ls_student-name.
  ENDLOOP.
ENDFORM.
```

（`tt_student` 是講義 4 用 TYPES 定義的表格型別（Table Type）——這就是表格型別的另一個好處：能拿來宣告參數。）

參數型別也可以直接用講義 25 在 SE11 建的 **DDIC Table Type**。例如讀取旺季加成設定的 FORM：

```
DATA gt_surchg TYPE ztr25_tt_surchg.

PERFORM load_surchg_config CHANGING gt_surchg.

FORM load_surchg_config CHANGING ct_surchg TYPE ztr25_tt_surchg.

```

```
  SELECT * FROM ztr25_surchg INTO TABLE ct_surchg.
ENDFORM.
```

兩種表格型別怎麼選：

型別來源	誰看得到	適合
`TYPES tt_xxx`（Local Type，講義 4）	只有這支程式	只在本程式內傳來傳去的表格
DDIC Table Type（講義 25）	全系統	多支程式要用同一種表格；**講義 15 的 FM 介面參數一定要用這種**

FORM 的參數型別兩種都可以；到了講義 15 的 Function Module 就沒得選——FM 介面參數只能用 DDIC 型別，用 `CHANGING` 搭配 DDIC Table Type 傳整張表，正是取代舊式 `TABLES` 參數的寫法（下一節）。

4. 區域變數（Local）vs 全域變數（Global）

- FORM 裡 `DATA` 宣告的是**區域變數**（`lv_`/`ls_`/`lt_`）：只在該 FORM 內存在，每次呼叫重新初始化。
- 程式開頭宣告的是**全域變數**（`gv_`/`gs_`/`gt_`）：所有事件與 FORM 都摸得到。

原則：**能區域就區域**。全域變數誰都能改，程式一大就追不到是誰改的；FORM 需要的資料盡量從參數進來，而不是伸手拿全域。傳統報表難免有核心全域內表（如 `gt_data`），但臨時變數絕不要全域。

5. 舊式 TABLES 參數（看得懂即可）

舊程式常見 `PERFORM f USING ... TABLES gt_x.` 或 `FORM f TABLES t_x STRUCTURE ...`——這是內表的舊式傳遞方式，**新程式不要用**（用 USING/CHANGING + 表格型別），但維護 ZDQM 等既有程式時要認得。Function Module 也有同樣的舊式 `TABLES` 參數，官方已標記為過時，講義 15 第 3.1、3.2 節會說明原因與替代寫法。

6. 傳統報表標準骨架

從本講開始，練習程式都照這個結構寫（也是期末實作與正式程式的長相）：

```
REPORT zr_xxx.

* 1) 宣告區：TYPES / DATA / CONSTANTS
* 2) 選擇畫面：PARAMETERS / SELECT-OPTIONS
* 3) 事件區：INITIALIZATION / START-OF-SELECTION ...
*    事件裡只放 PERFORM，不放邏輯
* 4) FORM 區：所有副程式，集中在檔尾
```

7. 常見錯誤與陷阱

症狀	原因
PERFORM 報「FORM 不存在」	FORM 名拼錯，或定義根本沒寫

症狀	原因
參數個數/順序錯	呼叫端與定義端依**位置**對應，一個都不能錯位
USING 的參數在 FORM 裡被改，呼叫端也變了	USING 預設傳參考——紀律：要改就宣告在 CHANGING
FORM 裡讀到莫名其妙的值	誤用了同名全域變數；區域變數記得 `l` 前綴
FORM 定義寫在事件中間，後面的事件不執行	FORM 區塊會「吃掉」後面的程式——FORM 一律放檔尾

8. 補充：跨程式呼叫 FORM 與 Subroutine Pool（選修，維護舊程式會遇到）

前面的 FORM 都是「同一支程式自己呼叫自己」。但語法也允許 **呼叫另一支程式裡的 FORM**——舊程式裡不少共用邏輯就是這樣寫的，所以要看得懂。

語法：`PERFORM ... IN PROGRAM`

```
PERFORM calc_grade IN PROGRAM zr_tr08_pool
  USING    gv_score
  CHANGING gv_grade.
```

Subroutine Pool（副程式池）：專門拿來放「給別人呼叫的 FORM」的程式，程式類型是 **Subroutine Pool**（SE38 建立時 Type 選這個，屬性代碼 `S`）：

```
PROGRAM zr_tr08_pool.          " 開頭是 PROGRAM，不是 REPORT

FORM calc_grade USING    iv_score TYPE i
                CHANGING cv_grade TYPE c.
  IF iv_score >= 80.
    cv_grade = 'A'.
  ELSEIF iv_score >= 60.
    cv_grade = 'B'.
  ELSE.
    cv_grade = 'C'.
  ENDIF.
ENDFORM.
```

- 只放 FORM：沒有選擇畫面（Selection Screen）、沒有事件、**不能 F8 直接執行**，只能被別的程式呼叫。
- 被呼叫時才載入記憶體（載入當下會觸發 `LOAD-OF-PROGRAM` 事件）。
- 技術上，可執行程式、Module Pool、Function Pool 裡的 FORM 也能被外部呼叫；Subroutine Pool 只是「最乾淨、專門為此設計」的容器。Include 裡的 FORM 不能用 Include 名稱直接呼叫。
- **建立方式是 GUI-only**：ADT 無法設定程式類型，要在 SE38 建立時選 Subroutine Pool（已建好的程式可用 Goto → Attributes 改 Type）。

`IF FOUND` 與動態指定：

```
* IF FOUND：找不到 FORM 或程式時安靜地跳過，不出錯
PERFORM no_such_form IN PROGRAM zr_tr08_pool IF FOUND
  USING gv_score CHANGING gv_grade.

* 動態指定：FORM 名與程式名放在變數裡，內容必須是大寫
DATA: gv_form TYPE c LENGTH 30 VALUE 'CALC_GRADE',
      gv_prog TYPE c LENGTH 30 VALUE 'ZR_TR08_POOL'.
PERFORM (gv_form) IN PROGRAM (gv_prog) IF FOUND
  USING gv_score CHANGING gv_grade.
```

沒加 `IF FOUND` 又找不到時，是**執行期例外**（已實測）：FORM 不存在是 `CX_SY_DYN_CALL_ILLEGAL_FORM`、程式不存在是 `CX_SY_PROGRAM_NOT_FOUND`——沒有攔截就 dump。

為什麼新程式不建議這樣寫（ABAP 官方文件的說法是外部呼叫「幾乎完全過時」）：

- **編譯時完全不檢查**：程式名、FORM 名是否存在、參數個數 / 順序 / 型別對不對，語法檢查都不管，全部要到執行期才爆；跟講義 8 前面強調的「參數加 `TYPE` 讓語法檢查幫你抓錯」正好相反。
- **難以追蹤**：被呼叫的程式會被載入呼叫端的同一個執行環境，影響範圍難以靜態判斷。
- **動態名稱有安全風險**：名稱若來自外部輸入，必須先檢查（系統類別 `CL_ABAP_DYN_PRG` 可用），否則等於讓外部決定要執行哪段程式。

跨程式共用邏輯的做法	介面在編譯時檢查	現況
FORM + Subroutine Pool	否	舊程式常見，新程式不用
Function Module（講義 15）	是	傳統報表的標準做法
Class Method（OOP 課程）	是	新世代開發的首選

練習見 ex08 的選修 Part（`ZR_TR08_POOL` + `ZR_TR08_CALLER`）。

9. 課堂練習

完成 ex08：把前幾講的學生報表重構成 get_data / process_data / display_data 三個 FORM，練習 USING 與 CHANGING。